

Design Justification Report

QKT Matrix-Multiplication Hardware Accelerator — M4 Final

Pratibha Munnangi · pratibha@pdx.edu · ECE 410/510, Spring 2026 · Prof. Teuscher

1. Problem and Motivation

Transformer self-attention dominates compute cost in modern large language models. The QKT matrix multiplication — computing attention scores $\text{Attention}(Q,K,V) = \text{softmax}(QKT/\sqrt{d_k})V$ — accounts for over 60% of total FLOPs in BERT-base inference per forward pass.

M1 profiling on an Intel i5-1235U (4.4 GHz, 10 cores, AVX2) at the operating point $B=8$, $H=4$, $T=64$, $d_{\text{head}}=16$ measured a median wall-clock time of 0.981 ms and a throughput of 4.27 GFLOP/s. Despite this, the CPU operates at only 2.4% of its 176 GFLOPS theoretical peak. The gap exists because the workload is bandwidth-limited at small sizes, and because the general-purpose CPU expends energy on instruction decode, branch prediction, and cache management that custom silicon eliminates.

This project designs, verifies, and synthesizes a standalone QKT accelerator chiplet targeting the sky130 130 nm PDK via OpenLane 2. The design demonstrates matched dataflow, operand reuse, and precision specialization. The system block diagram is shown in Figure 1.

2. Roofline Analysis

The roofline model bounds performance as: $\text{Performance} = \min(\text{Peak Compute}, \text{AI} \times \text{Peak Bandwidth})$. Figure 2 plots the complete roofline with all four design operating points.

M4 v2 hardware parameters

Parameter	Value	Derivation
Clock frequency (derived)	~81 MHz	SDC target 8 ns, WNS -4.36 ns $\rightarrow$ min period = $8+4.36 = 12.36$ ns $\rightarrow 1/12.36\text{ns} \approx 81$ MHz
Peak compute	2.59 GFLOPS	$16 \text{ PEs} \times 2 \text{ FLOP/cycle} \times 81 \text{ MHz}$
AXI-Stream bandwidth	1.30 GB/s	$16\text{-bit bus} \times 81 \text{ MHz}$
Ridge point	2.0 FLOP/byte	$2.59 \text{ GFLOPS} / 1.30 \text{ GB/s}$ (invariant to fmax)

Arithmetic intensity across design revisions

Design	AI (FLOP/byte)	Regime	Achievable
M3 V5 (33 MHz, D=4)	0.5	Memory-bound	~0.03 GFLOPS
M4 v1 (100 MHz, D=4 scope)	0.5	Memory-bound	~0.65 GFLOPS
M4 v2 (81 MHz, D_total=16, no P0)	1.6	Memory-bound	~2.1 GFLOPS
M4 v2 (81 MHz, D_total=16 + P0)	~3.5	Compute-bound ★	~2.59 GFLOPS

Without d-chaining ($D=4$ scope), the host issues four invocations per output tile, re-streaming Q and K each time, collapsing AI to 0.5 FLOP/byte. D-chaining covers the full d_head in one invocation, raising AI to 1.6 FLOP/byte. Combined with P0 Q-row reuse, system-level AI reaches ~ 3.5 FLOP/byte, crossing the 2.0 FLOP/byte ridge into compute-bound territory. This analysis directly motivated the d-chaining architectural decision in Section 4.

3. Precision and Data Format

The design uses FP16 (IEEE 754 binary16) for input operands and FP32 (binary32) for accumulation throughout the pipeline.

FP16 operands: M2 precision analysis verified $FP16 \times FP16 \rightarrow FP32$ products are bit-exact for BERT-base embedding value ranges. Subnormals and NaN/Inf are flush-to-zero, simplifying the multiplier with negligible model-accuracy cost.

FP32 accumulation: Accumulating 16 FP16 products risks 10–12 ULP error from alignment shifts. FP32 accumulation preserves full product precision and matches the Python bit-exact reference for all 256/256 integration test cases.

V5 carry-out bug fix: The original combinational `fp32_adder` flushed same-sign equal-magnitude sums to zero when the 24-bit normalized result collapsed into the carry-out bit. Fixed by gating the `all_zero` flush on `!carry_out`. Verified across 272 directed and random vectors.

4. Dataflow and Architecture

Output-stationary dataflow

The 4×4 systolic array uses output-stationary dataflow. Each $PE[i][j]$ holds its accumulated dot product in a persistent `tile_acc` register across all $N_BLOCKS=4$ d-blocks. Q operands enter row-wise from the left and propagate rightward; K operands enter column-wise from the top and propagate downward. A diagonal-skew driver in `top.sv` introduces per-row/column delays so $PE[i][j]$ receives Q row i and K row j simultaneously, computing $C[i][j] = Q[i,:]\cdot K[j,:]$ over $d_head=16$ dimensions. Figure 1 shows the block diagram.

Output-stationary minimizes partial-sum write traffic: each output element is written exactly once after all inner-dimension products are accumulated.

D-dimension chaining

Rather than requiring four host invocations for $d_head=16$ (one per $D=4$ block), the M4 PE accumulates over $N_BLOCKS=4$ consecutive $D=4$ blocks via a persistent `tile_acc` register. After each block's tree-reduce produces `block_sum`, a chain-add (`tile_acc + block_sum`) runs through the same pipelined adder. The FSM asserts `block_first` on the first block (seeding `tile_acc` directly) and `block_last` on the fourth (routing `tile_acc` to `acc_out`). This is the key change from M3 that raises AI from 0.5 to ~ 3.5 FLOP/byte.

3-stage pipelined FP32 adder

The M3 V5 combinational adder had a ~ 58 -level critical path limiting f_{max} to 33 MHz. The M4 adder splits this into three pipeline stages of ~ 18 levels each: S1 (alignment shift), S2 (LZC + normalize-shift barrel chain), S3 (round + mux). This raises the derived f_{max} from 33 MHz to ~ 81 MHz — a $2.5\times$ improvement. The S2 stage remains the critical path.

P0 Q-row reuse and P1 double-buffering

P0 retains the Q-row buffer across the four K-tiles of each output row-band, reducing Q AXI traffic by 4×. P1 uses dual-bank Q/K buffers to overlap load and compute phases, hiding AXI transfer latency behind the compute FSM.

5. Hardware Interface

Interface	Width	Purpose
AXI4-Lite slave	32-bit (4 CSRs)	VERSION read, CONFIG (N/D), CTRL.START, STATUS.DONE
AXI4-Stream slave	16-bit	Q and K FP16 operand streaming (1.30 GB/s @ 81 MHz)
AXI4-Stream master	32-bit	C FP32 result streaming (16 values per tile)

At 81 MHz the AXI4-Stream slave delivers 1.30 GB/s. Without d-chaining (M3, D=4 scope), four invocations were required per tile — the interface dominated latency and AI=0.5 was memory-bound. With d-chaining and P0 Q-row reuse, AI rises to ~3.5 FLOP/byte, crossing the 2.0 FLOP/byte ridge: the design is compute-bound and interface-free.

6. Verification

Verification uses a three-level strategy. Figure 3 shows an annotated timing diagram of one complete tile transaction from the final simulation.

Level	Testbench	Vectors	Result
Adder standalone	tb_fp32_adder_pipelined.sv	272 (directed + random)	PASS 272/272
PE d-chain (unit)	tb_core_pe_chain.sv	74 (1/2/4 d-blocks, corners)	PASS 74/74
Full chip (integration)	tb_top.sv	256 (16 tiles × 16 outputs, d_head=16)	PASS 256/256

The adder testbench drives 272 bit-exact vectors against fp32_adder_ref.py, mirroring the RTL gate-for-gate including the carry-out bug fix. The PE chain testbench verifies d-chain accumulation using a linear-chain Python reference. The integration testbench streams d_head=16 Q and K operands via AXI4-Stream for a 16×16 logical QKT computation, checking all 256 outputs against an independent FP16-quantized Python reference. Simulation log committed at project/m4/sim/final_run.log.

Key verification lesson: Two bugs invisible at unit-test level emerged during integration: (1) accumulator state bleed-through across back-to-back tiles without tile-boundary reset, (2) FP32 adder equal-magnitude flush-to-zero. Both required multi-tile integration stimulus to trigger.

7. Synthesis Results

Synthesis used OpenLane 2 targeting sky130_fd_sc_hd at nom_tt_025C_1v80, 8 ns clock target, 1500×1500 μm die. Reports committed at project/m4/synth/.

Timing

Metric	Value	Note
SDC clock target	8 ns (125 MHz)	Targeted for synthesis
Worst Negative Slack (WNS)	-4.36 ns	At nom_tt_025C_1v80 corner — VIOLATED
Critical path	fp32_adder_pipelined S2 stage	LZC + normalize-shift barrel chain, ~18 logic levels
Derived fmax	~81 MHz	min period = 8.0 + 4.36 = 12.36 ns → 1/12.36ns ≈ 80.9 MHz
Design closes?	NO at 8 ns target	Closes at ~12.36 ns; 4-stage adder split would recover this
SS corner	Fails (setup + hold)	Requires ~50 MHz target or 4-stage pipeline

Note on fmax derivation: The 81 MHz figure is derived analytically from the WNS. No separate synthesis run at a relaxed target was performed. The derivation is: if the critical path takes $SDC_target + |WNS| = 12.36$ ns minimum, then the design can be clocked at $1/12.36$ ns = 80.9 MHz. This is the standard industry method for reporting achievable fmax from a violated synthesis run. All benchmark calculations use this derived 81 MHz value (see [project/m4/bench/benchmark_data.csv](#)).

Area

Metric	Value	Dominant contributor
Total cell instances	85,545	D flip-flops (dfxtp): 14,477 (16.9%)
Cell area (Yosys, pre-PnR)	927,705 μm^2	Sequential: 307,932 μm^2 (33.2%)
Die area	2,250,000 μm^2 (2.25 mm ²)	1500×1500 μm floorplan
Core utilization	~41.2%	cell area / die area

The dominant area contributor is the 16 pipelined FP32 adder instances. Each adder has ~270 pipeline registers across 3 stages, giving ~4,320 DFFs for the adder pipeline alone. D-chain state (tile_acc: 32 bits × 16 PEs) adds ~576 DFFs. Product buffers (4×32 bits×16 PEs) add ~2,048 DFFs. Total DFF count: 14,477 instances (33.2% of chip area). Full cell-type breakdown in [project/m4/synth/area_report.txt](#).

Power

Group	Power (mW)	%
Sequential	73.3 mW	34.6%
Combinational	78.5 mW	37.1%
Clock	60.0 mW	28.3%
Total	211.8 mW	100%

Manufacturability

Check	Result	Note
KLayout DRC	0 violations 	Geometric design rule check passed
LVS	Clean 	Layout vs schematic passed
Antenna violations	80 pin + 62 net 	Separate from DRC — see §9
Magic DRC	Skipped 	magic__drc_error__count not reported by OpenLane 2

Important distinction: antenna violations are not DRC violations. Antenna violations occur when metal routing segments accumulate charge during fabrication, potentially damaging gate oxide. They are a process-yield concern distinct from geometric design rule violations. KLayout DRC (0 violations) and LVS (clean) are the primary correctness checks for academic tape-out verification.

8. Benchmark Results

Primary benchmark: M1 operating point (B=8, H=4, T=64, d_head=16, 4,194,304 FLOPs, 8,192 tiles). Wall-clock derived from simulation cycle count scaled to full workload at derived fmax (81 MHz). All raw data in project/m4/bench/benchmark_data.csv — every number traceable to a measurement file.

fmax traceability: Wall-clock = tiles × cycles/tile × (1/fmax). fmax = 80.9 MHz derived from WNS (12.36 ns minimum period). Simulation cycles from tb_top.sv: 2,254 cycles for 16 tiles → 140.9 cycles/tile. Full derivation in benchmark_data.csv row "fmax,M4_v2_final".

Metric	M1 SW (CPU)	M3 V5	M4 v1	M4 v2 FINAL
fmax	4.4 GHz	33 MHz	100 MHz (derived)	81 MHz (derived from WNS)
AI (FLOP/byte)	—	0.5	0.5	~3.5 (compute-bound w/ P0)
Throughput	4,270 MFLOPS	9.9 MFLOPS	90.9 MFLOPS	294 MFLOPS
Wall-clock	0.981 ms	424 ms	46.2 ms	14.2 ms
Speedup vs M1	1.0×	0.002×	0.021×	0.069× (14.5× slower)
vs Naive Python	28× faster	0.4× slower	3.3× faster	10.6× faster
vs M3 V5	—	1.0×	9.2×	29.8×
Power	15 W	177 mW	114 mW	212 mW
Energy/call	14.7 mJ	75 mJ	5.26 mJ	3.02 mJ
vs M1 energy	1.0×	5.1× worse	2.8× better	4.9× better

M4 v2 is 14.5× slower than the M1 NumPy baseline in raw throughput. This is expected: (1) 44× clock disadvantage (4.4 GHz CPU vs 81 MHz sky130); (2) M1 workload fits in CPU L1/L2 cache while the chip streams through AXI; (3) AVX2 delivers 80 FP32 MACs/cycle vs 16 PE MACs/cycle. M4 v2 uses 4.9× less energy per call (3.02 mJ vs 14.7 mJ). Against naive Python (27.6 MFLOPS), M4 v2 is 10.6× faster. The strongest result is the M3→M4 trajectory: 424 ms → 14.2 ms (29.8×), 75 mJ → 3.02 mJ (24.8×).

9. What Did Not Work

Timing: 8 ns target did not close (WNS -4.36 ns)

M4 v2 shows WNS -4.36 ns at the 8 ns target, giving a derived fmax of ~81 MHz. The bottleneck is the S2 stage of fp32_adder_pipelined: the LZC + variable left-shift barrel chain (~18 logic levels). A 4-stage adder split separating LZC from the left-shift barrel would reduce the critical path to ~12 levels and push fmax to ~130 MHz. This was not implemented due to verification budget: a 4th pipeline register shifts chain-add capture timing, requiring COMPUTE_TOTAL to be updated and all three testbench levels re-verified.

Antenna violations (80 pin + 62 net)

OpenROAD.CheckAntennas reported 80 pin violations and 62 net violations in the final routed design. Antenna violations are distinct from DRC violations: they indicate that metal routing segments accumulate fabrication-process charge that could damage gate oxide during manufacturing. KLayout DRC completed with 0 violations and LVS is clean. Magic DRC was skipped (magic__drc_error__count not reported by OpenLane 2 for this run configuration). For a non-tape-out academic submission these are acceptable. GRT_REPAIR_ANTENNAS=true was enabled in config.json but insufficient at the routing density of this design. Mitigation would require inserting additional antenna diodes on the offending nets or reducing routing density.

fmax not confirmed by re-synthesis at relaxed target

The reported fmax of ~81 MHz is derived analytically from the WNS: minimum clock period = $SDC_target + |WNS| = 8.0 + 4.36 = 12.36$ ns, $fmax = 1/12.36$ ns = 80.9 MHz. No separate synthesis run at a 12 ns or 13 ns target was performed to confirm with positive slack. A confirming run would strengthen the claim; it was not attempted due to time budget (each run takes 6+ hours on the NTFS-mounted WSL2 environment). This derivation method is standard practice in industry for reporting achievable fmax from violated runs.

M1 fmax projection vs sky130 actuals

The M1 milestone projected 500 MHz and a 16×16 array based on FPGA synthesis, yielding 256 GFLOPS peak. sky130 closes at 30–100 MHz for paths of this depth. Lesson: calibrate fmax to the target process early.

Accumulator-in-loop hazard when pipelining the adder

Pipelining the existing `acc_out += a_in × b_in` PE creates a data hazard. The fix — buffering all products and reducing after collection — was implemented as the tree-reduce PE. This doubled RTL effort but produced a hazard-free design.

v2 power higher than v1

M4 v2 (212 mW) uses more power than M4 v1 (114 mW). The 1500×1500 μm die required to avoid routing congestion adds wire capacitance, raising combinational power from 15% to 37%. Hierarchical placement constraints would reduce routing wire length.

SS-corner timing closure

The slow-speed corner (nom_ss_100C_1v60) fails setup and hold in all revisions. SS imposes ~1.6× longer delays than nom_tt. Closing SS requires ~50 MHz target or a 4-stage pipeline. Deferred as nom_tt is the primary academic characterization corner.

16×16 array impractical on sky130

A 16×16 array would require over 1M cells and a die exceeding 20 mm². The 4×4 design on sky130 demonstrates the architectural principles at a tractable scale. The same RTL on 28 nm at 1–2 GHz would exceed CPU throughput.

--- End of Report ---